

Reviewer Pack — Minimal Rank of Geometric Langlands Lattices Bounds XOR-AND ...

Entry 1dd07b7d478d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 17:37:33 UTC

Minimal Rank of Geometric Langlands Lattices Bounds XOR-AND Tree Width

Entry ID: 1dd07b7d478d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 17:37:33 UTC

1. Conjecture

Field A (mathematical branch): Geometric Langlands Theory **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

[‘For every geometric Langlands lattice associated with a compact Riemann surface, the minimal rank of its generalized flag variety is upper-bounded by a function that depends polynomially on the size of the XOR-AND tree for the corresponding Boolean function.’, ‘Specifically, if L is a geometric Langlands lattice and T is the XOR-AND tree width of the associated Boolean function, then the minimal rank of L , denoted as $\kappa(L)$, satisfies $\kappa(L) = O(T^2)$.’, ‘For all $n \leq 40$, if an instance has an XOR-AND tree width greater than some threshold $C(n)$, then the corresponding geometric Langlands lattice has a minimal rank greater than a function of $C(n)$.’]

Rationale (proposer’s reasoning):

[‘Geometric Langlands theory has deep connections to representation theory and algebraic geometry, which may provide insights into complexity lower bounds. The study of lattices in this theory is less explored compared to other areas, offering a novel angle.’, ‘XOR-AND tree width is a known lower bound for various complexity measures. Investigating its relationship with the rank of geometric Langlands lattices could lead to new complexity-theoretic results.’, ‘This conjecture bridges algebraic geometry and complexity theory in a way that has not been extensively explored, potentially revealing underlying structures that are relevant to computational hardness.’]

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 24d50f54611dbabf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if Spearman's rank correlation coefficient (ρ) is ≥ 0.8 for at least 80% of the 30 seeds, and the mean $\kappa(L)/T^2$ ratio across all seeds is ≤ 1 . For falsification, any seed results in $\rho < 0.5$ or a $\kappa(L)/T^2$ ratio greater than 1.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "geometric Langlands theory" AND "XOR-AND tree width" AND bounds - "minimal rank" Geometric Langlands lattice "polynomial bound" XOR-AND tree width - "generalized flag variety" geometric Langlands lattice "function of T^2 " XOR-AND tree width

Top relevant hits considered: - [http://arxiv.org/abs/0710.4339v1] Heavy-Quark Masses from the Fermilab Method in Three-Flavor Lattice QCD - [http://arxiv.org/abs/1111.0981v1] Form factors for B to $K\ell\ell$ semileptonic decay from three-flavor lattice QCD - [http://arxiv.org/abs/1308.1705v1] Rotating lattice

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def xor_and_tree_width(boolean_function):
    if isinstance(boolean_function, tuple) and len(boolean_function) == 2:
        left, right = boolean_function
        return 1 + max(xor_and_tree_width(left), xor_and_tree_width(right))
    elif isinstance(boolean_function, int):
        return 0
    else:
        raise ValueError("Invalid Boolean function")

def calculate_metric(boolean_function):
    T = xor_and_tree_width(boolean_function)
    if T == 0:
        return 0, 0
    rank = len(boolean_function) ** (1 / T)
    return rank, T

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

n = random.randint(5, 40)
boolean_function = tuple(random.choice([0, 1]) for _ in range(n))

try:
    metric_value, T = calculate_metric(boolean_function)
    instances_tested = 1
    conjecture_holds = (metric_value / T**2 <= 1)
    counterexample = "" if conjecture_holds else f"n={n}, rank={metric_value}, T={T}"
except Exception as e:
    return {
        "seed": seed,
        "metric_name": "rank/T^2",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": str(e)
    }

return {
    "seed": seed,
    "metric_name": "rank/T^2",
    "metric_value": metric_value / T**2,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(100, 999) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if not results:
        print("RESULT: INCONCLUSIVE no trials run")
    else:
        total_metric_value = sum(r["metric_value"] * r["instances_tested"] for r in results)
        instances_tested = sum(r["instances_tested"] for r in results)
        mean_metric_value = total_metric_value / instances_tested
        std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 * r["instances_tested"] for r in results) / instances_tested)

        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

        if all(r["conjecture_holds"] for r in results):
            print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
        elif any(not r["conjecture_holds"] for r in results) and sum(1 for r in results if not r["conjecture_holds"]) < 3:
            print(f"RESULT: FALSIFIED counterexample={counterexample}")
        else:
            print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction} < 0.8")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
Invalid Boolean function'}
TRIAL: {'seed': 593, 'metric_name': 'rank/T^2', 'metric_value': None, 'instances_tested': 0, 'conjecture': 'rank/T^2'}
TRIAL: {'seed': 631, 'metric_name': 'rank/T^2', 'metric_value': None, 'instances_tested': 0, 'conjecture': 'rank/T^2'}
TRIAL: {'seed': 677, 'metric_name': 'rank/T^2', 'metric_value': None, 'instances_tested': 0, 'conjecture': 'rank/T^2'}
TRIAL: {'seed': 727, 'metric_name': 'rank/T^2', 'metric_value': None, 'instances_tested': 0, 'conjecture': 'rank/T^2'}
TRIAL: {'seed': 773, 'metric_name': 'rank/T^2', 'metric_value': None, 'instances_tested': 0, 'conjecture': 'rank/T^2'}
TRIAL: {'seed': 821, 'metric_name': 'rank/T^2', 'metric_value': None, 'instances_tested': 0, 'conjecture': 'rank/T^2'}
TRIAL: {'seed': 877, 'metric_name': 'rank/T^2', 'metric_value': None, 'instances_tested': 0, 'conjecture': 'rank/T^2'}
TRIAL: {'seed': 929, 'metric_name': 'rank/T^2', 'metric_value': None, 'instances_tested': 0, 'conjecture': 'rank/T^2'}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_085ab850.py", line 75, in <module>
    total_metric_value = sum(r["metric_value"] * r["instances_tested"] for r in results)
                        ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_085ab850.py", line 75, in <genexpr>
    total_metric_value = sum(r["metric_value"] * r["instances_tested"] for r in results)
                        ~~~~~^~~~~~
TypeError: unsupported operand type(s) for *: 'NoneType' and 'int'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's validity. | next: Review and debug the test code to ensure it can run successfully and produce meaningful results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12763
2	preregistration	ollama_remote	glm4:latest	0	0	6111
3	novelty	ollama_remote	glm4:latest	0	0	4932
4	novelty	ollama_remote	glm4:latest	0	0	5841
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13816
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9781
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10017
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9408
9	judge	ollama_remote	glm4:latest	0	0	8307

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 80977 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/1dd07b7d478d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1dd07b7d478d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1dd07b7d478d.tar.gz` (if generated)